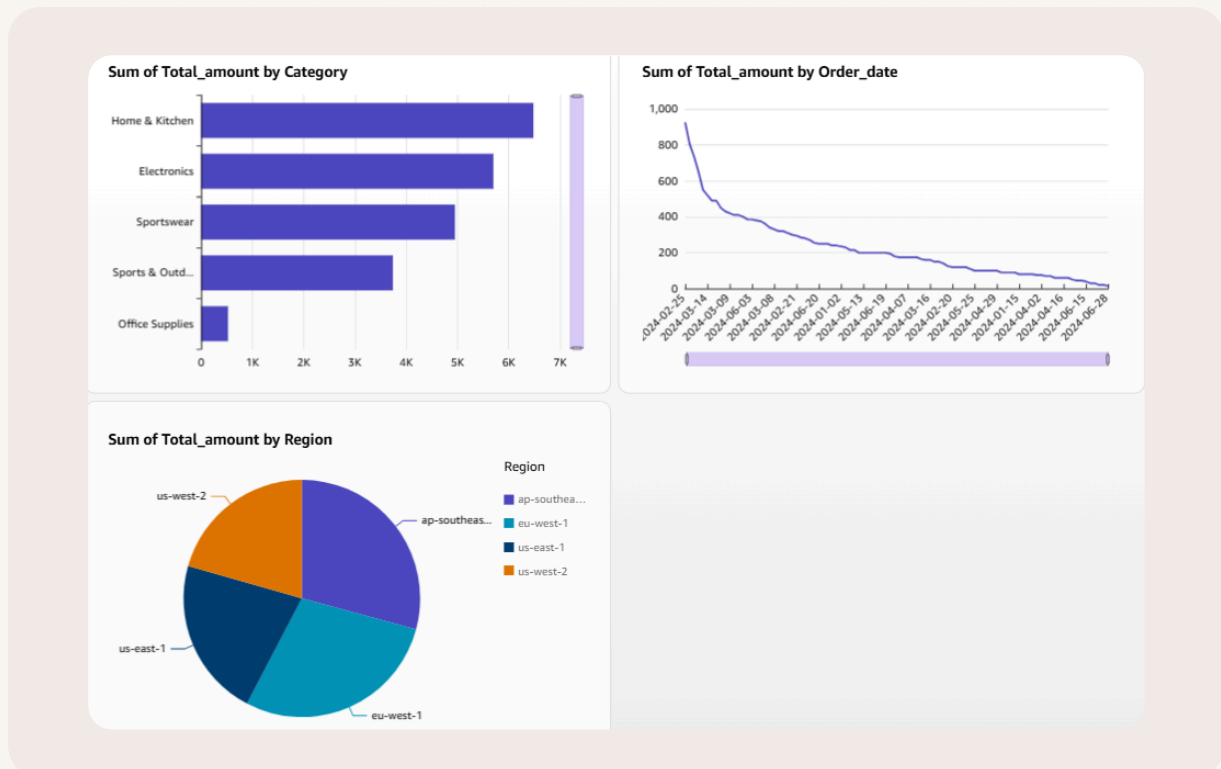


SERVERLESS E-COMMERCE ANALYTICS DASHBOARD

By Nikhila Reddy | May 2026



Project Vision: Serverless Analytics on AWS

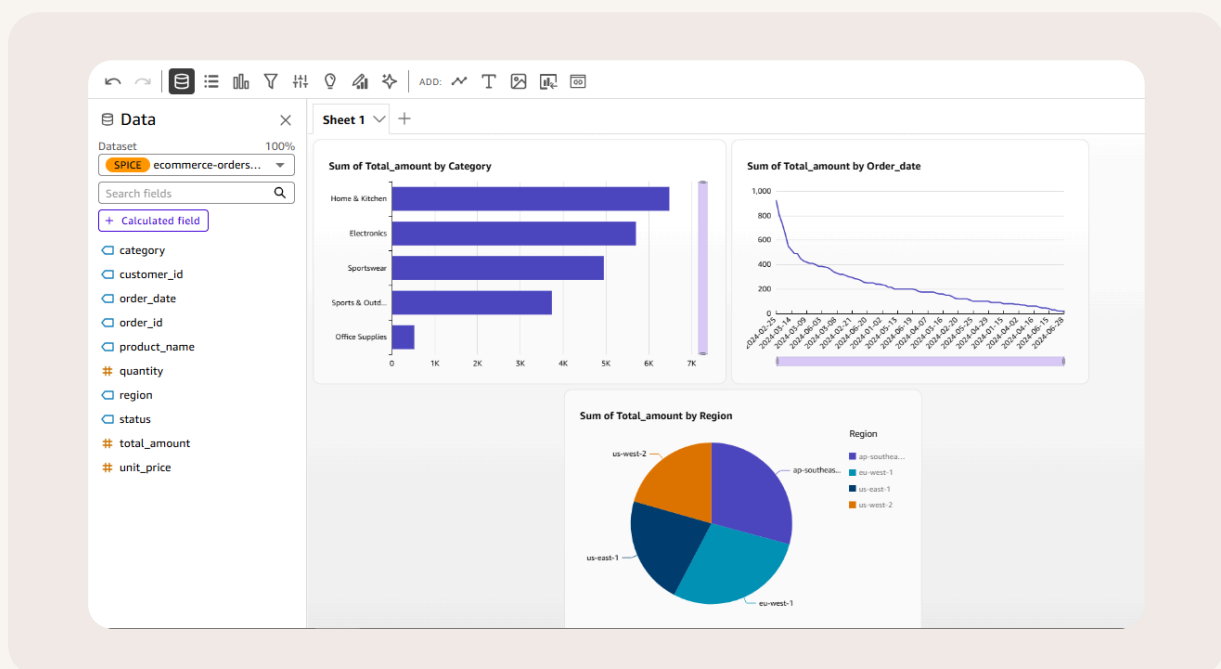
What this pipeline achieves

In this project, I'm building a serverless e-commerce analytics pipeline using AWS services like Lambda, DynamoDB, S3, Athena, and QuickSight. The goal is to generate sample business data, process and store it in cloud-based data layers, query it using SQL, and visualize insights through interactive dashboards to better understand modern cloud-native analytics and business reporting workflows.

Building the QuickSight Dashboard

Connecting Athena to QuickSight

In this step, I'm connecting Amazon QuickSight to the Athena dataset and building an interactive analytics dashboard from the e-commerce order data. I'll import the dataset into SPICE, create visualizations for revenue trends, order activity, and regional sales performance, and then publish the analysis as a shareable business dashboard.



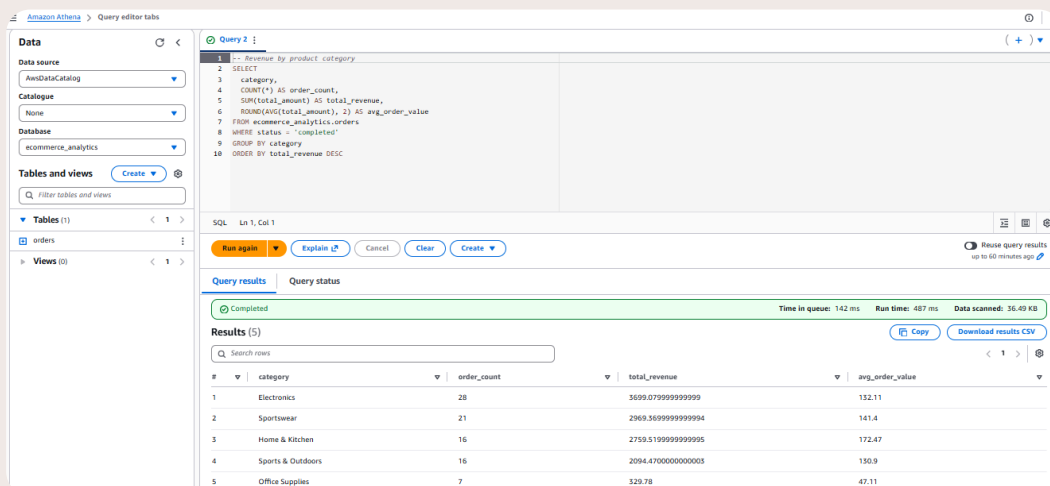
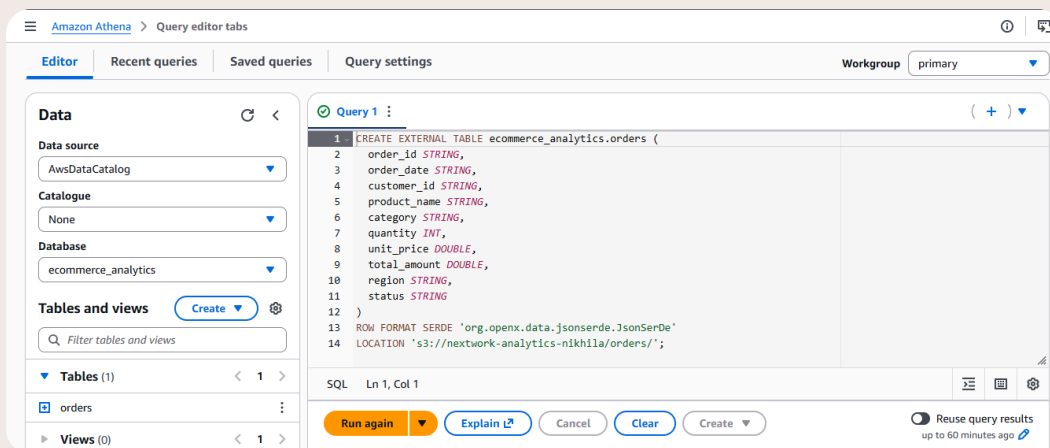
Business insights from each visualization

The bar chart answers which product categories are bringing in the most revenue. The line chart shows how sales change over time and helps spot overall order trends. The pie chart reveals how revenue is spread across different regions and makes it easier to compare regional sales performance.

Querying the Data Lake with Athena

Setting up the Glue database and external table

In this step, I'm using Amazon Athena to query the e-commerce order data stored in S3 using SQL. I'll configure Athena, create a Glue database and external table to define the structure of the JSON data, and then run analytics queries to uncover business insights such as revenue trends, customer activity, and regional performance.



Deploying the Lambda Data Pipeline

Writing and packaging the Lambda function

In this step, I'm creating and deploying a Lambda function that generates sample e-commerce order data and stores it in both DynamoDB and S3. I'll also create an IAM role to give the function the required permissions and then use the AWS CLI to package, deploy, and test the serverless data pipeline within AWS.

```
}
~ $ aws iam create-role --role-name ecommerce-lambda-role --assume-role-policy-document file://trust-policy.json
{
  "Role": {
    "Path": "/",
    "RoleName": "ecommerce-lambda-role",
    "RoleId": "ARO330RA7FJA436WZRV",
    "Arn": "arn:aws:iam::814871542098:role/ecommerce-lambda-role",
    "CreateDate": "2026-05-13T09:09:38+00:00",
    "AssumeRolePolicyDocument": {
      "Version": "2012-10-17",
      "Statement": [
        {
          "Effect": "Allow",
          "Principal": {
            "Service": "lambda.amazonaws.com"
          },
          "Action": "sts:AssumeRole"
        }
      ]
    }
  }
}
```

```
~ $ aws lambda create-function --function-name ecommerce-data-loader --zip-file fileb://function.zip --handler lambda_function.lambda_handler --runtime python3.12 --role arn:aws:iam::814871542098:role/ecommerce-lambda-role --timeout 60
{
  "FunctionName": "ecommerce-data-loader",
  "FunctionArn": "arn:aws:lambda:us-east-1:814871542098:function:ecommerce-data-loader",
  "Runtime": "python3.12",
  "Role": "arn:aws:iam::814871542098:role/ecommerce-lambda-role",
  "Handler": "lambda_function.lambda_handler",
  "CodeSize": 1522,
  "Description": "",
  "Timeout": 60,
  "MemorySize": 128,
  "LastModified": "2026-05-13T09:36:12.473+0000",
  "CodeSha256": "qunhbmimr3UEAQIDW7HCFs1KX7M6cKE2o6A8K51+",
  "Version": "$LATEST",
  "TracingConfig": {
    "Mode": "PassThrough"
  },
  "RevisionId": "8d1bd52b-187e-4694-94f7-09e1f2154ab0",
  "State": "Pending",
  "StateReason": "The function is being created.",
  "StateReasonCode": "Creating",
  "PackageType": "Zip",
  "Architecture": [
    "x86_64"
  ],
  "ephemeralStorage": {
    "Size": 512
  },
  "SnapStart": {
    "ApplyType": "None",
    "OptimizationStatus": "OFF"
  },
  "RuntimeVersionConfig": {
    "RuntimeVersionArn": "arn:aws:lambda:us-east-1:runtime:eb4b553846c4a081813ff7216098a5358d50956b0b081c83c662a31:"
  }
}
```

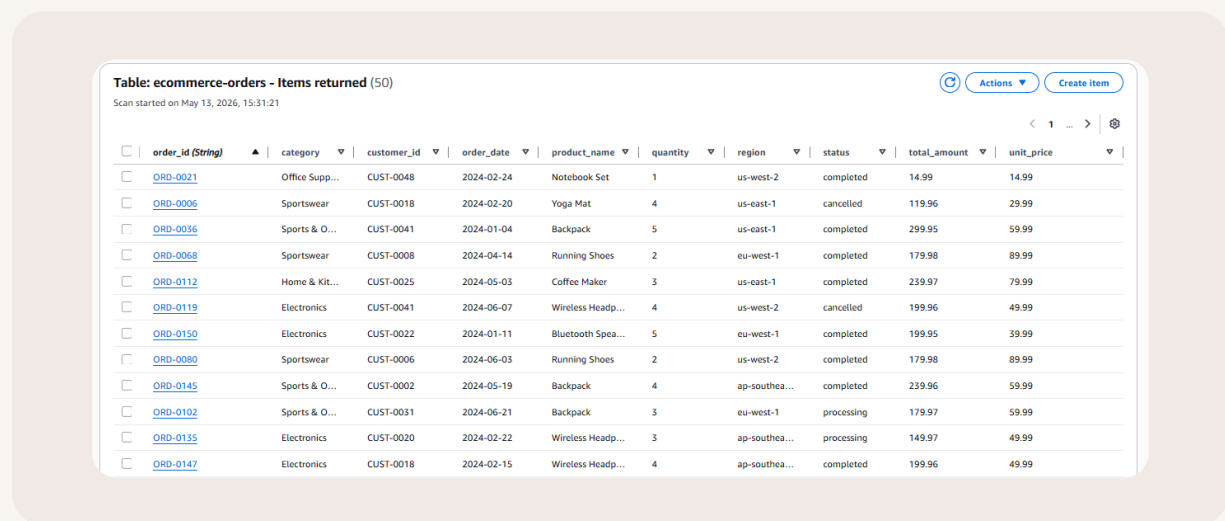
Dual-write pattern: DynamoDB and S3

The Lambda function writes data to both DynamoDB and Amazon S3. DynamoDB is used as the operational data store for storing structured e-commerce order records that can be accessed quickly, while S3 is used as the analytics storage layer so Athena and QuickSight can query, analyze, and visualize the data for reporting and dashboard creation.

Verifying the Data Pipeline End-to-End

Invoking Lambda and confirming output

In this step, I'm testing the serverless data pipeline by invoking the Lambda function and verifying that the sample e-commerce order data is successfully being written to both DynamoDB and S3. This confirms that the operational storage and analytics layers are working correctly before moving on to querying and visualization.



The screenshot displays the AWS Data Explorer console for a table named 'ecommerce-orders'. The table contains 50 items, with the first 15 visible. The columns are: order_id (String), category, customer_id, order_date, product_name, quantity, region, status, total_amount, and unit_price. The data shows various orders with different statuses like 'completed', 'cancelled', and 'processing'.

order_id (String)	category	customer_id	order_date	product_name	quantity	region	status	total_amount	unit_price
ORD-0021	Office Supp...	CUST-0048	2024-02-24	Notebook Set	1	us-west-2	completed	14.99	14.99
ORD-0006	Sportswear	CUST-0018	2024-02-20	Yoga Mat	4	us-east-1	cancelled	119.96	29.99
ORD-0036	Sports & O...	CUST-0041	2024-01-04	Backpack	5	us-east-1	completed	299.95	59.99
ORD-0068	Sportswear	CUST-0008	2024-04-14	Running Shoes	2	eu-west-1	completed	179.98	89.99
ORD-0112	Home & Kit...	CUST-0025	2024-05-03	Coffee Maker	3	us-east-1	completed	239.97	79.99
ORD-0119	Electronics	CUST-0041	2024-06-07	Wireless Headp...	4	us-west-2	cancelled	199.96	49.99
ORD-0150	Electronics	CUST-0022	2024-01-11	Bluetooth Spea...	5	eu-west-1	completed	199.95	39.99
ORD-0080	Sportswear	CUST-0006	2024-06-03	Running Shoes	2	us-west-2	completed	179.98	89.99
ORD-0145	Sports & O...	CUST-0002	2024-05-19	Backpack	4	ap-southea...	completed	239.96	59.99
ORD-0102	Sports & O...	CUST-0031	2024-06-21	Backpack	3	eu-west-1	processing	179.97	59.99
ORD-0135	Electronics	CUST-0020	2024-02-22	Wireless Headp...	3	ap-southea...	processing	149.97	49.99
ORD-0147	Electronics	CUST-0018	2024-02-15	Wireless Headp...	4	ap-southea...	completed	199.96	49.99

Order record structure and DynamoDB data types

Each order record contains fields such as order ID, order date, customer ID, product name, category, quantity, unit price, total amount, region, and order status. The price-related fields are stored as Numbers so DynamoDB can handle them correctly as numeric values for calculations, filtering, and analytics instead of treating them as plain text.

Objects (1)

  Copy S3 URI  Copy URL  Download  Open  Delete

Actions

Create folder

Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

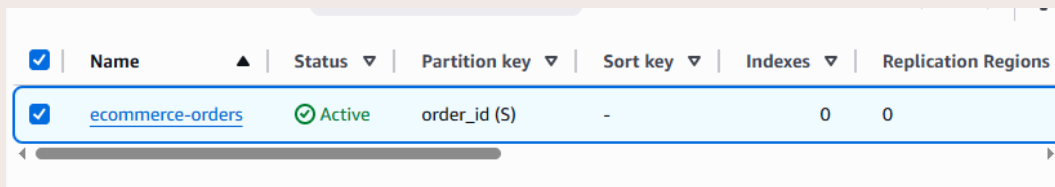
< 1 > 

☐	Name ▲	Type ▼	Last modified ▼
☐	 orders.json	json	May 13, 2026, 15:28:12 (UTC+05:30)

Provisioning the AWS Infrastructure

S3, DynamoDB, and QuickSight setup

In this step, I'm setting up the foundational AWS resources for a serverless analytics pipeline so that the Lambda function can generate and store e-commerce data in DynamoDB and S3, while Athena and QuickSight will later be used to query, analyze, and visualize business insights through interactive dashboards.



A screenshot of the AWS IAM console showing a table of roles. The table has columns for Name, Status, Partition key, Sort key, Indexes, and Replication Regions. A single role named 'ecommerce-orders' is listed with a status of 'Active', a partition key of 'order_id (S)', and no sort key, indexes, or replication regions.

☒	Name	Status	Partition key	Sort key	Indexes	Replication Regions
☒	ecommerce-orders	Active	order_id (S)	-	0	0

Role of each resource in the pipeline

In this step, I created an S3 bucket, a DynamoDB table, and activated Amazon QuickSight. The S3 bucket will store analytics data files and Athena query results for querying and reporting, DynamoDB will store the operational e-commerce order records generated by the Lambda function, and QuickSight will later be used to analyze and visualize the data through interactive business dashboards.

Secret Mission: Interactive Filters and KPI Metrics

Date range filter and headline KPI visuals

I added three KPI metrics to display total revenue, total orders, and average order value so the dashboard could show key business insights at a glance. I also added Start Date and End Date filter controls that allow users to select a custom date range and dynamically update all the charts and KPI visuals across the dashboard.

Reflections and Key Takeaways

Tools and concepts mastered

In this project, I learned how to build a serverless analytics pipeline using AWS services including Lambda, DynamoDB, S3, Athena, Glue, and QuickSight. I gained hands-on experience with Python-based Lambda functions, IAM roles and permissions, deploying resources with the AWS CLI, querying S3 data using Athena and SQL, and building interactive dashboards in QuickSight. I also learned concepts such as operational vs analytics storage layers, JSON Lines data formatting, serverless data ingestion, and cloud-based business reporting workflows.

Time and challenges

This project took me approximately 120 minutes to complete, including creating the documentation. The most challenging part was working with AWS CLI and CloudShell while creating and managing the required JSON configuration files for IAM roles, permissions, and Lambda testing. Since the project instructions assumed local file creation, I had to adapt the workflow for CloudShell and better understand how serverless deployment and file-based commands work in AWS.

I did this project today to learn how to build an end-to-end serverless analytics workflow on AWS using Lambda, DynamoDB, S3, Athena, and QuickSight. Another skill I want to learn is how to build more advanced serverless applications using API Gateway and cloud automation workflows.